

Práctica: Servicios de Aplicación, DTOs y CRUD

Paginado

Paso a paso — Exponer endpoints REST con DTOs, validaciones de negocio, filtros y paginación en base de datos

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 5 — Repositorios, Servicios y CRUD
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Crear DTOs inmutables con records (CrearSolicitudDto, ActualizarSolicitudDto, SolicitudDto, PagedResult<T>) para proteger el modelo de dominio.
- Implementar la capa de aplicación con SolicitudService orquestando la lógica de negocio y el Unit of Work.
- Construir SolicitudesController exponiendo operaciones CRUD semánticas (GET paginado, GET por ID, POST, PUT, DELETE).
- Enriquecer el controlador con un endpoint analítico especializado GET /api/solicitudes/resumen-estados para tableros de control.
- Validar el ciclo de vida completo de la API REST mediante pruebas en Swagger UI y archivo .http con verificación de códigos de respuesta HTTP.

2. Desarrollo paso a paso

Paso 1 — Declarar los DTOs inmutables del caso de uso y PagedResult

En la capa Application definimos los modelos de transferencia de datos mediante records inmutables, desacoplando los contratos de la API del esquema relacional y evitando vulnerabilidades de Over-Posting:

Archivo 1: SolicitudesApp.Application/DTOs/PagedResult.cs — Contenedor genérico para respuestas paginadas con propiedades calculadas para navegación:

Application/DTOs/PagedResult.cs

```
namespace SolicitudesApp.Application.DTOS;

public record PagedResult<T>(
    IReadOnlyList<T> Items,
    int TotalItems,
    int PaginaActual,
    int TamanoPagina,
    int TotalPaginas
)
{
    public bool TienePaginaAnterior => PaginaActual > 1;
    public bool TienePaginaSiguiente => PaginaActual < TotalPaginas;
}
```

Archivo 2: SolicitudesApp.Application/DTOs/SolicitudDtos.cs — Modelos inmutables de entrada y salida para el flujo de solicitudes:

Application/DTOs/SolicitudDtos.cs

```
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.DTOS;

public record CrearDetalleDto(string Concepto, int Cantidad, decimal PrecioUnitario);

public record CrearSolicitudDto(
    string Motivo,
    int SolicitanteId,
    int? CentroCostoId,
    List<CrearDetalleDto> Detalles
);

public record ActualizarSolicitudDto(
    string Motivo,
    int? CentroCostoId
);

public record SolicitudDetalleDto(
    int Id,
    string Concepto,
    int Cantidad,
    decimal PrecioUnitario,
    decimal Importe
);

public record SolicitudDto(
    int Id,
    stringCodigo,
    string Motivo,
    decimal MontoTotal,
    EstadoSolicitud Estado,
    string EstadoNombre,
    int SolicitanteId,
    string SolicitanteNombre,
    int? CentroCostoId,
    string CentroCostoNombre,
    DateTime FechaCreacion,
    List<SolicitudDetalleDto> Detalles
);

public record ResumenEstadosDto(
    string Estado,
    int Cantidad,
    decimal MontoTotal
);
```

Inmutabilidad y seguridad con Records: los tipos record en C# proveen igualdad por valor y propiedades inmutables por defecto. Al no exponer las entidades de dominio directamente en la API evitamos serializar referencias circulares y prevenimos la asignación de campos protegidos como Id, Estado o MontoTotal por parte de clientes externos.

Paso 2 — Diseñar el contrato ISolicitudService y la orquestación en SolicitudService

En Application/Services/ implementamos la lógica de negocio del caso de uso. El contrato define las operaciones públicas y la clase concreta orquesta la creación calculando los importes, validando reglas de negocio y coordinando con IUnitOfWork:

Archivo 1: SolicitudesApp.Application/Services/ISolicitudService.cs — Contrato de los casos de uso del módulo de Solicitudes:

Application/Services/ISolicitudService.cs

```
using SolicitudesApp.Application.DTOs;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.Services;

public interface ISolicitudService
{
    Task<SolicitudDto> CrearAsync(CrearSolicitudDto dto, CancellationToken ct = default);
    Task<SolicitudDto?> ObtenerPorIdAsync(int id, CancellationToken ct = default);
    Task<PagedResult<SolicitudDto>> ListarPaginadoAsync(int pagina, int tamanoPagina,
        EstadoSolicitud? estado = null, CancellationToken ct = default);
    Task ActualizarAsync(int id, ActualizarSolicitudDto dto, CancellationToken ct = default);
    Task EliminarAsync(int id, CancellationToken ct = default);
    Task<IReadOnlyList<ResumenEstadosDto>> ObtenerResumenPorEstadosAsync(CancellationToken ct =
        default);
}
```

Archivo 2: SolicitudesApp.Application/Services/SolicitudService.cs — Implementación del servicio de aplicación con orquestación y validaciones de negocio:

Application/Services/SolicitudService.cs

```
using SolicitudesApp.Application.DTOS;
using SolicitudesApp.Application.Interfaces;
using SolicitudesApp.Domain.Entidades;
using SolicitudesApp.Domain.Excepciones;

namespace SolicitudesApp.Application.Services;

public class SolicitudService : ISolicitudService
{
    private readonly IUnitOfWork _uow;
    public SolicitudService(IUnitOfWork uow) => _uow = uow;

    public async Task<SolicitudDto> CrearAsync(CrearSolicitudDto dto, CancellationToken ct = default)
    {
        if (dto.Detalles == null || !dto.Detalles.Any())
            throw new ReglaDominioException("La solicitud debe tener al menos un ítem de detalle.");

        var solicitud = new Solicitud
        {
            Motivo = dto.Motivo,
            SolicitanteId = dto.SolicitanteId,
            CentroCostoId = dto.CentroCostoId
        };

        foreach (var d in dto.Detalles)
            solicitud.Detalles.Add(new SolicitudDetalle
            {
                Concepto = d.Concepto,
                Cantidad = d.Cantidad,
                PrecioUnitario = d.PrecioUnitario
            });

        solicitud.RecalcularTotal();
        await _uow.Solicitudes.AddAsync(solicitud, ct);
        await _uow.SaveChangesAsync(ct);

        var recargada = await _uow.Solicitudes.GetByIdConDetallesAsync(solicitud.Id, ct);
        return MapearADto(recargada!);
    }

    public async Task<SolicitudDto?> ObtenerPorIdAsync(int id, CancellationToken ct = default)
    {
        var s = await _uow.Solicitudes.GetByIdConDetallesAsync(id, ct);
        return s is null ? null : MapearADto(s);
    }

    public async Task<PagedResult<SolicitudDto>> ListarPaginadoAsync(int pagina, int tamanoPagina, EstadoSolicitud? estado = null, CancellationToken ct = default)
    {
        var pagedEntities = await _uow.Solicitudes.ObtenerPaginadoAsync(pagina, tamanoPagina, estado, ct);
        var dtos = pagedEntities.Items.Select(MapearADto).ToList();
        return new PagedResult<SolicitudDto>(dtos, pagedEntities.TotalItems, pagedEntities.PaginaActual, pagedEntities.TamanoPagina, pagedEntities.TotalPaginas);
    }

    public async Task ActualizarAsync(int id, ActualizarSolicitudDto dto, CancellationToken ct = default)
    {
        var solicitud = await _uow.Solicitudes.GetByIdAsync(id, ct)
            ?? throw new KeyNotFoundException($"No se encontró la solicitud con Id {id}");

        if (solicitud.Estado != EstadoSolicitud.Borrador)
            throw new ReglaDominioException("Solo se pueden editar solicitudes en estado Borrador.");

        solicitud.Motivo = dto.Motivo;
        solicitud.CentroCostoId = dto.CentroCostoId;
        await _uow.Solicitudes.UpdateAsync(solicitud, ct);
        await _uow.SaveChangesAsync(ct);
    }

    public async Task EliminarAsync(int id, CancellationToken ct = default)
    {

```

```
var solicitud = await _uow.Solicitudes.GetByIdAsync(id, ct)
?? throw new KeyNotFoundException($"No se encontró la solicitud con Id {id}");

if (solicitud.Estado != EstadoSolicitud.Borrador)
    throw new ReglaDominioException("Solo se pueden eliminar solicitudes en estado Borrador.");

await _uow.Solicitudes.DeleteAsync(solicitud, ct);
await _uow.SaveChangesAsync(ct);
}

public async Task<IReadOnlyList<ResumenEstadosDto>>
ObtenerResumenPorEstadosAsync(CancellationToken ct = default)
{
    var todas = await _uow.Solicitudes.ListAllAsync(ct);
    return todas
        .GroupBy(s => s.Estado)
        .Select(g => new ResumenEstadosDto(g.Key.ToString(), g.Count(), g.Sum(s =>
s.MontoTotal)))
        .ToList();
}

private static SolicitudDto MapearADto(Solicitud s) => new(
    s.Id, $"SOL-{s.Id:D5}", s.Motivo, s.MontoTotal, s.Estado, s.Estado.ToString(),
    s.SolicitanteId, s.Solicitante?.Nombre ?? "N/A",
    s.CentroCostoId, s.CentroCosto?.Nombre ?? "N/A", s.FechaCreacion,
    s.Detalles.Select(d => new SolicitudDetalleDto(d.Id, d.Concepto, d.Cantidad,
d.PrecioUnitario, d.Importe)).ToList()
);
}
```

Reglas de negocio protegidas en Application: las comprobaciones de estado (por ejemplo, validar que solo se modifiquen o cancelen solicitudes en estado Borrador) residen en la capa de negocio, protegiendo la integridad del ciclo de vida antes de solicitar la persistencia.

Paso 3 — Construir SolicitudesController con endpoints RESTful y códigos HTTP semánticos

En Api/Controllers/SolicitudesController.cs creamos el controlador delgado (thin controller) inyectando ISolicitudService. Exponemos el CRUD completo y el endpoint analítico enriquecido con códigos de respuesta HTTP estrictos (200, 201, 204, 400, 404).

Api/Controllers/SolicitudesController.cs

```
using Microsoft.AspNetCore.Mvc;
using SolicitudesApp.Application.DTOS;
using SolicitudesApp.Application.Services;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Api.Controllers;

[ApiController]
[Route("api/[controller]")]
public class SolicitudesController : ControllerBase
{
    private readonly ISolicitudService _service;

    public SolicitudesController(ISolicitudService service) => _service = service;

    [HttpGet]
    public async Task<ActionResult<PagedResult<SolicitudDto>>> GetAll(
        [FromQuery] int pagina = 1,
        [FromQuery] int tamanoPagina = 10,
        [FromQuery] EstadoSolicitud? estado = null)
    {
        var result = await _service.ListarPaginadoAsync(pagina, tamanoPagina, estado);
        return Ok(result);
    }

    [HttpGet("{id:int}")]
    public async Task<ActionResult<SolicitudDto>> GetById(int id)
    {
        var item = await _service.ObtenerPorIdAsync(id);
        if (item == null) return NotFound(new { mensaje = $"No se encontró la solicitud con Id {id}" });
        return Ok(item);
    }

    [HttpPost]
    public async Task<ActionResult<SolicitudDto>> Create([FromBody] CrearSolicitudDto dto)
    {
        var creada = await _service.CrearAsync(dto);
        return CreatedAtAction(nameof(GetById), new { id = creada.Id }, creada);
    }

    [HttpPut("{id:int}")]
    public async Task<IActionResult> Update(int id, [FromBody] ActualizarSolicitudDto dto)
    {
        await _service.ActualizarAsync(id, dto);
        return NoContent();
    }

    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Delete(int id)
    {
        await _service.EliminarAsync(id);
        return NoContent();
    }

    [HttpGet("resumen-estados")]
    public async Task<ActionResult<IReadOnlyList<ResumenEstadosDto>>> GetResumenEstados()
    {
        var resumen = await _service.ObtenerResumenPorEstadosAsync();
        return Ok(resumen);
    }
}
```

Semántica REST de códigos HTTP: un POST exitoso devuelve 201 Created junto a la cabecera 'Location' apuntando a la URL del nuevo recurso (/api/solicitudes/{id}). Las modificaciones (PUT) o eliminaciones (DELETE) exitosas que no requieren contenido de retorno devuelven 204 No Content.

Paso 4 — Configurar la API e inyección de dependencias en Program.cs

En Api/Program.cs conectamos los servicios de infraestructura mediante la extensión `AddInfrastructureServices`, habilitamos Swagger OpenAPI y aseguramos la creación de la base de datos al inicio de la aplicación.

Api/Program.cs

```
using SolicitudesApp.Infrastructure;
using SolicitudesApp.Infrastructure.Persistencia;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
??
"Server=localhost;Database=SolicitudesDb;Trusted_Connection=True;TrustServerCertificate=True";

// Registro de Infrastructure: DbContext, Unit of Work, Repositorios y Servicios
builder.Services.AddInfrastructureServices(connectionString);

var app = builder.Build();

// Inicialización de la base de datos de desarrollo
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<SolicitudesDbContext>();
    db.Database.EnsureCreated();
}

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Creación automática en desarrollo: `Database.EnsureCreated()` genera las tablas y aplica la semilla de datos del ModelBuilder sin necesidad de ejecutar comandos CLI adicionales durante la sesión de laboratorio.

Paso 5 — Validar el ciclo CRUD completo y paginación con solicitudes.http

Utilizamos el cliente REST integrado en Visual Studio o VS Code mediante un archivo `.http` para ejecutar secuencialmente la creación, consulta paginada, actualización y eliminación de solicitudes verificando cada código de estado.

solicitudes.http

```
@baseUrl = http://localhost:5000

### 1. Crear una nueva solicitud con múltiples detalles (Esperado: 201 Created)
POST {{baseUrl}}/api/solicitudes
Content-Type: application/json

{
  "motivo": "Adquisición de periféricos y monitores para equipo TI",
  "solicitanteId": 1,
  "centroCostoId": 1,
  "detalles": [
    { "concepto": "Monitor Dell 27 pulg 4K", "cantidad": 2, "precioUnitario": 1250.00 },
    { "concepto": "Teclado mecánico ergonómico", "cantidad": 2, "precioUnitario": 180.50 }
  ]
}

### 2. Obtener la solicitud creada por Id (Esperado: 200 OK)
GET {{baseUrl}}/api/solicitudes/1
Accept: application/json

### 3. Listado paginado con filtro de estado (Esperado: 200 OK + PagedResult)
GET {{baseUrl}}/api/solicitudes?pagina=1&tamanoPagina=5&estado=Borrador
Accept: application/json

### 4. Actualizar solicitud en estado Borrador (Esperado: 204 No Content)
PUT {{baseUrl}}/api/solicitudes/1
Content-Type: application/json

{
  "motivo": "Adquisición de monitores TI - Ajustado por cotización final",
  "centroCostoId": 2
}

### 5. Consultar resumen analítico agrupado por estado (Esperado: 200 OK)
GET {{baseUrl}}/api/solicitudes/resumen-estados
Accept: application/json

### 6. Eliminar solicitud en estado Borrador (Esperado: 204 No Content)
DELETE {{baseUrl}}/api/solicitudes/1

### 7. Verificar eliminación de la solicitud (Esperado: 404 Not Found)
GET {{baseUrl}}/api/solicitudes/1
Accept: application/json
```

Pruebas reproducibles con archivos .http: los archivos .http documentan las llamadas API de manera ejecutable dentro del IDE, permitiendo validar variables de entorno, cabeceras y payloads JSON sin salir del entorno de desarrollo.

3. Retos adicionales

1. Implementa un filtro adicional en GET /api/solicitudes para buscar solicitudes por texto en el motivo (usando .Contains() en el repositorio).
2. Crea un middleware global o filtro de excepciones para capturar KeyNotFoundException (retornando 404) y ReglaDominioException (retornando 400 Bad Request) automáticamente.
3. Agrega un endpoint PATCH /api/solicitudes/{id}/motivo para permitir actualizar únicamente el motivo sin exigir el centro de costo.
4. Escribe una prueba de integración usando WebApplicationFactory<Program> para simular la petición POST /api/solicitudes y validar que retorne status 201.

4. Entregables

- Modelos de DTOs inmutables implementados con records y clase genérica `PagedResult<T>`.
- Servicio `SolicitudService` implementando validaciones de estado antes de actualizar o eliminar.
- Controlador `SolicitudesController` exponiendo los 6 endpoints (CRUD completo + paginación + resumen de estados).
- Archivo `solicitudes.http` documentando la ejecución exitosa de las peticiones HTTP y sus respuestas semánticas.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 5 · Docente: Edwin Barrientos · New Horizons